TAKE-HOME PROJECT · TECHNICAL DEEP DIVE

StockMaxxer

A quant portfolio with an LLM risk overlay, replayed one real trading day at a time on real prices and real SEC filings.

Vimalkumar Patel

Java 21 · Spring Boot · SQLite · Next.js · Claude

THE THESIS

Price models see that a stock moved. They cannot see that the company rewrote its own risk factors.

Prior work

Lazy Prices (Cohen, Malloy, Nguyen, 2020): firms that change their filings tend to underperform. It measures **how much** the text changed.

This project

An LLM reads only the **new** Item 1A sentences and says **what** changed: a named customer or a quantified exposure, not boilerplate.

The constraint

The model proposes, the code decides. The LLM can only **trim** a position, never raise one. It is a risk overlay, not a return forecaster.

THE BRIEF

What the brief asked for

01	AI as a core constraint: something rules alone cannot do	02	Prices plus one unstructured source (SEC 10-K / 10-Q)
03	BUY / SELL / HOLD and concrete weights at all times	04	Verifiable reasoning, in plain language, for every line
05	A forecast over a declared horizon	06	Deterministic and replayable for any historical slice
07	A backtester decoupled from the engine it tests	08	End-to-end audit, no black boxes, runs on a clean machine

TRACEABILITY

Each requirement, where it lives, honest status

Requirement	Where in the code	Status
Prices + unstructured data	Yahoo, Edgar, Filings: real bars and 10-K/10-Q	Met
AI as a core constraint	Signals + Llm: Claude scores new risk sentences	Met
BUY / SELL / HOLD, weights	Engines.target: weights with CASH sum to 1	Met
Verifiable reasoning	Verified quote + template explanation per line	Met
Forecast, declared horizon	Forecast: 63-day block bootstrap, fixed seed	Met
Deterministic replay	Simulated clock + sha256-keyed LLM cache	Met
Decoupled backtester	Backtest takes any engine id, read-only	Met
Full audit incl. prompts	Append-only audit; LLM answer by cache key	Simplified
Historical alpha check	Backtest + filing-level eval; sample too small	No alpha claim
Clean-machine run	JDK 21, Maven, Node; keys for the first fetch	Simplified

ARCHITECTURE

Independent jobs around one database



Claude sits behind a cache

Only the signals job calls it. Every answer is stored in data/llm-cache/<sha256>.json.

Jobs share only the DB

Each job re-runs safely. A failing job exits non-zero and stops the night.

3 Maven modules

core, app, jobs. The web app and the jobs stay separate runnables.

A SIMULATED NIGHT

One night moves the clock to the next real trading day



The recommendation lifecycle

PROPOSED → accept → **ACCEPTED** → next open → **FILLED**

PROPOSED → ignore → **IGNORED**

PROPOSED → the day ends → **EXPIRED**

Holdings change only at FILLED, at the next day's open.

One recommendation per portfolio per day. Accept only on its own day, else HTTP 409.

THE DECISION CORE · 1

Point-in-time by construction, not by a guard

Two tables

history holds every downloaded bar. bars holds only the days revealed so far.

Engines read only bars

Day D+1 does not exist yet for an engine, a backtest or an eval.

Filings count from the next day

Never too early, and no time zones.

```
// Market.java · the prices job reveals one day
INSERT OR IGNORE INTO bars
  SELECT symbol, trade_date, open, close
  FROM history WHERE trade_date = ?

// every engine read
SELECT close FROM bars
  WHERE symbol = ? AND trade_date <= ?
  ORDER BY trade_date DESC LIMIT ?
```

THE DECISION CORE · 2

A real quant base, so the A/B is fair

Rank

The benchmark's 12 largest holdings by 20-day price change. Keep the top 8.

Weight

In proportion to 1/σ of daily returns, scaled to 98%. The other 2% is the cash floor.

Cap

15% per name. The excess is spread over the rest, repeated until nothing is over.

```
// Engines.java
static final int LOOKBACK = 20;
static final int TOP_N = 8;
static final BigDecimal INVESTED = new BigDecimal("0.98");
static final BigDecimal NAME_CAP = new BigDecimal("0.15");

// quantOnly(): weight ∝ 1/volatility
inverse.put(s.symbol(),
    BigDecimal.ONE.divide(s.vol(), MC));

// Names over the 15% cap are pinned at it and
// the excess is spread over the rest. It repeats,
// because spreading can push another name over.
```

THE DECISION CORE · 3

The model proposes, the code decides

```
// Engines.trimmed()
int s = flag == null ? 0 : flag.severity();
trim.put(e.getKey(), e.getValue()
    .multiply(TRIM_PER_SEVERITY)
    .multiply(BigDecimal.valueOf(s)));
BigDecimal room = CASH_CAP.subtract(BigDecimal.ONE
    .subtract(sum(base.values()))).max(BigDecimal.ZERO);
BigDecimal scale = totalTrim.compareTo(room) > 0
    ? room.divide(totalTrim, MC) : BigDecimal.ONE;
BigDecimal w = e.getValue()
    .subtract(trim.get(e.getKey()).multiply(scale, MC));
// Rounded UP: still never above the base weight
weights.put(e.getKey(),
    w.setScale(WEIGHT_SCALE, RoundingMode.UP));
```

Trim = w × severity × 10%

Severity 5 halves a weight. Severity 0 changes nothing.

Trimmed weight → cash

Cash is capped at 15%. Every trim scales down pro-rata, and the scale is audited.

Never raises a weight

The LLM can express risk, never conviction.

THE DECISION CORE · 4

From a filing to a severity

1. **Store** every 10-K and 10-Q since 2024 for 35 stocks, complete: about 380 filings.
2. **Cut** Item 1A, Risk Factors, from the normalized text.
3. **Diff** against the previous filing of the same form. A sentence is new when fewer than half of its 5-word runs appeared before.
4. **Ask Claude** about only the new sentences: up to 10 risks, each a closed type, severity 0–5, scope and quote. Nothing new: no call.
5. **Score** day D with the highest verified firm-scope event in the latest filing before D.

S	Meaning (a fixed, uncalibrated rubric)
0	No change, or industry-wide boilerplate
1	Firm-specific rewording, no new risk
2	New risk, generic, no named exposure
3	New risk with a named customer, supplier, country or regulator
4	New firm-specific risk, quantified or material
5	Existential: going concern, restatement, material weakness

TRUST · 1

Every quote is checked word for word

```
// Signals.java
static boolean verified(String quote, String item1a) {
    String q = Filings.normalizeText(quote);
    return !q.isEmpty() && item1a.contains(q);
}


-- only these events can move a weight
AND e.scope = 'firm'
AND e.verified = 1
AND e.severity > 0
```

One text space

The check and the UI use the same normalized text.

Paraphrase is kept, never used

It is stored with verified = 0, so it can never move a weight.

Industry news is dropped

Tariffs or export rules would trim a whole sector.

NVDA: 80 of 80 quotes verified

TRUST · 2

Determinism comes from a cache, not a temperature

```
// Llm.java
public String key(String prompt) {
    return sha256(model + "\n"
        + Prompts.VERSION + "\n" + prompt);
}

if (replayOnly) {
    throw new IllegalStateException("replay only
      (SM_LLM_REPLAY): no cached answer " + key);
}

// Only a complete answer may be cached.
if (!stopReason.equals("end_turn")) {
    throw new IllegalStateException(…"not cached");
}
```

Same inputs, same answer

A cached answer is always reused. A second run makes no API calls.

Versioned by key

A new model or prompt version is a new key: a clean re-score.

Fail loudly

A refused answer would replay forever as "no risks", which looks like good news.

TRUST · 3

Money is exact, and every buy is a lot

BigDecimal

Stored as TEXT. JSON sends decimals as strings.

Cash is the residual

Weights with CASH sum to exactly 1.

Shares round down

So cash is never negative.

FIFO lots

Sell the oldest first. A lot's price never changes.

```
// Portfolios.resize(): a pure function
BigDecimal toSell = held.subtract(target);
for (Lot lot : oldestFirst) {
  if (toSell.signum() == 0) {
    result.add(lot);
  } else if (lot.shares()
      .compareTo(toSell) <= 0) {
    toSell = toSell.subtract(lot.shares());
  } else {
    result.add(new Lot(symbol, lot.acquiredOn(),
      lot.shares().subtract(toSell), lot.price()));
    toSell = BigDecimal.ZERO;
  }
}
```

TRUST · 4

An append-only trail for every recommendation

Kind	What the row records
PROPOSED	Engine, universe, prompt version, a sha256 of every close it could read, prior and target weights, trim scale, each trim's filing, quote, cache key and model
ACCEPTED, IGNORED, EXPIRED	The row itself records when
FILLED	Each trade at the open, and the cash after

Written in the same transaction as the change it records.

```
// Audit.java · not in schema.sql: Spring
// splits that file at every ';'
for (String action :
        List.of("UPDATE", "DELETE")) {
  db.sql("CREATE TRIGGER IF NOT EXISTS"
    + " audit_no_" + action.toLowerCase()
    + " BEFORE " + action + " ON audit"
    + " BEGIN SELECT RAISE(ABORT, 'the"
    + " audit trail is append-only'); END")
    .update();
}

// test: replayingAProposalGives
//       TheWeightsItRecorded
```

EXPLAINABLE TO A NON-SPECIALIST

Templates, not prose from the model

> *"MU is number 5 of 12 stocks for price change in the last 20 trading days (+14.4%), so it is in the top 8. Its price moves less each day than most of the other top stocks, so it gets a larger part. The system buys more: its part changes from 13.72% to 13.75%."*

ASD-STE100 rules

Short sentences, active voice, one idea each. A test checks the length and the count.

Same numbers as the trade

So an explanation can never disagree with the decision.

The quote is the evidence

A trim line shows the verified filing sentence under it.

MEASUREMENT

Forecast, backtest, eval, and no overclaiming

Forecast

63 trading days. A 5-day block bootstrap, 10,000 paths, seed 42, over the last 252 days at today's weights.

p10 / p50 / p90, labelled "historical dispersion, not a prediction"

Backtest

Takes any engine id. Fills at the next open, marks at the close, as if every recommendation were accepted.

End NAV, return, max drawdown, average cash, vs buy-and-hold

Eval

Severity vs the next 20 days' return minus the 35-stock mean. One row per filing, never per event.

Also: quote verification rate, share of industry events

⚠ **No performance numbers here, on purpose.** Only NVDA's filings are scored so far. A few hundred filings are underpowered, so the system claims no alpha.

JUDGMENT

Simplifications, each with its reason

Real-world design	Here	Why
Wall clock, 09:30 ET cut-off	A simulated day in the DB	Same rule, no time zones, no fake clock
11 Maven modules	3: core, app, jobs	Web app and jobs stay separate; split later
Universe screen (SIC, ADV)	Benchmark's 12 largest names	A rule, not a hand-pick; survivorship stated
Corporate-action prices	Split-adjusted, no dividends	Far less code; price returns for all
Spring Data repositories	JdbcClient + plain SQL	Fewer lines than mapping keys and decimals
Audit hash chain	Append-only triggers	Tamper-evidence: a question nobody asked

Each one is marked **Simplification:** in the code and listed in the README, 25 rows in all.

LIVE DEMO · OFFLINE

Ten real trading days, five seconds each

What to watch

1. Day D arrives; D+1 does not exist yet
2. A trim with its verified filing quote
3. Accepted today, filled at tomorrow's open
4. Two QQQ portfolios drift apart: Claude vs quant only
5. One recommendation's audit trail

```
$ scripts/demo.sh 10 5
======== 2026-01-07 ========
QQQ with Claude (quant_llm, QQQ)
  filled at today's open: SELL 0.051523
    AMD @ 212.1300; ... BUY 1.308971 WMT
  NAV $998.12, cash 6.50%
  QQQ from the same start: $1017.78
  today's recommendation (ACCEPTED):
    NVDA BUY 6.77% -> 8.14% severity 4:
    "On September 15, 2025, China's
    antitrust regulators ..."
```

sample output from the README · the fallback

LIMITATIONS AND NEXT STEPS

What is missing, and what I would do next

Known limits

- Severity saturates at 4: most held names get trimmed
- No co-occurrence backstop for sector-wide news
- Fixed universe with survivorship bias
- Price returns only: no dividends or corporate actions
- No beta, alpha or information ratio yet
- Realised gains are visible only as cash

Next

1. Score all 35 stocks; publish the eval with standard errors
2. A severity aggregation that does not saturate
3. The co-occurrence backstop from the Python design
4. A rule-based universe screen, point-in-time
5. Postgres and scheduled jobs in a container

APPENDIX A

More simplifications

Real-world design	Here	Why
Exchange holiday calendar	The days SPY traded	The real calendar comes with the data
Fetch day D on day D	Fetch day D on D+1	An intraday bar would freeze a wrong close
σ_{60}, 12-month momentum, top 15	Top 8 by 20-day return, $1/\sigma_{20}$	Same shape; a month of history is enough
Sector cap, no-trade band	Not applied	12 names are too few for a sector cap
Absorbing severity 5, decay, clusters	Max firm-scope severity, latest filing	Short and explainable
Raw prompt + answer in the audit	Cache key + model	Keeps the database small
Next.js + TypeScript + charts	One plain-JSX page	Fewest files; /api is proxied, no CORS
Beta, alpha, IR	Equity vs the rebased benchmark	No statistics code

APPENDIX B

The API, base path /api/v1

Method	Path	Returns or does
POST	/portfolios	Create: name, deposit, benchmark, engine
GET	/portfolios/{id}	Cash, NAV, holdings, cost basis
DELETE	/portfolios/{id}	Archive (soft delete); /restore undoes it
POST	/portfolios/{id}/recommendations	Today's recommendation, now
GET	/portfolios/{id}/decisions	Lines, explanation, quote
POST	…/decisions/{d}/accept | ignore	409 if not allowed now
GET	…/decisions/{d}/audit	The trail, oldest first
GET	/portfolios/{id}/equity	NAV vs the rebased benchmark
GET	/portfolios/{id}/forecast	63-day p10, p50, p90
GET	/benchmarks, /engines, /clock	Allowed values, simulated day

Decimals travel as JSON strings. Errors: 400 bad input, 404 not found, 409 not allowed now. No auth: a local, personal app.

APPENDIX C

Likely questions

What does the LLM cost?

About 170 calls score every filing once. Replays cost nothing; only new filings add calls.

How does it scale?

Signals are per filing and cached. 35 stocks is a choice. SQLite goes first.

SQLite to Postgres?

Plain SQL. Change INSERT OR IGNORE, the triggers, and TEXT decimals to NUMERIC.

Swap the model?

Set SM_LLM_MODEL. A new key gives a clean re-score; old answers stay for comparison.

What if a job fails?

It exits non-zero and stops the night. Every job re-runs safely.

Why Java?

Prototyped with Python and Claude, and took the lessons to a well written app on Spring Boot stack with separation of concerns.